

TranceVR: De la Dungeon Procedural la Randare Fractală în WebXR

Studiu de caz: optimizare, eșecuri tehnice și procesare de semnal

Buciu Theodor, Țacu Darius Ștefan

24 mai 2026

- **Ipoteză:** Utilizarea editorului vizual (Babylon.js Editor) pentru compoziția rapidă a scenei 3D.
- **Realitate:** Lipsă totală de optimizare pentru scene cu instanțieri complexe.
- **Problema (The 3 FPS Crash):** Overhead masiv în graful scenei. Fiecare nod vizual introducea latență, blocând Render Loop-ul.
- **Concluzie:** Editorul a fost abandonat complet. S-a decis trecerea la **Cod Pur (TypeScript)**

- **Motorul Core:** Utilizarea `@babylonjs/core` instanțiat manual, eliminând abstractizările inutile.
- **Fizica:** Integrarea motorului **Havok Physics** (bibliotecă WASM ultra-performantă) pentru simularea precisă și rapidă a coliziunilor (Ground/Player).

Eșecul #2 – Conceptul "Infinite Dungeon"

- **Ideea:** Un coridor infinit creat procedural din 3-4 module realizate în Blender.
- **Mecanica:** Concatenare dinamică via LevelStreamingSystem.
- **Punctul critic:** Shadere audio-reactive aplicate direct pe pereții fizici.
- **Cauza eșecului:** Incompatibilitate între materialele PBR din '.glb' și injecția de cod GLSL vertex shader și pipeline-ul babylon.js. Scăderi drastice de framerate și artefacte vizuale (Z-fighting).

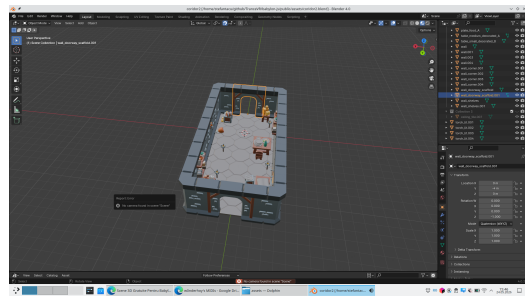


Figura: Module Blender - Vedere top-down coridoare .glb

Pivotul Final – Minimalizarea Scenei

- **Decizie Arhitecturală:** Abandonarea coridoarelor geometrice și a sistemului de streaming.
- **Noua Viziune:** O singură cameră VR într-un spațiu infinit, generat matematic pe ecran.
- **Design "Ochelari de Cal":** Randăm totul într-un fragment shader (Fractal Raymarching), eliminând complet geometria 3D tradițională.
- **Mecanica Vizuală:** Trecerea la un **Global Screen-Space Shader**. Elimină definitiv problemele de adâncime și costurile de instanțiere.

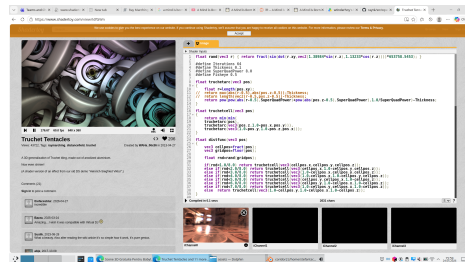


Figura: Rezultat Final: Shader Fractal Reactiv

- **Tehnologie:** Web Audio API (AnalyserNode) integrat în AudioService.
- **Fast Fourier Transform (FFT):** Transformarea semnalului din domeniul timpului în frecvență. Rezoluție: 512 unități (bins).
- **Smoothing:** Factor de 0.6 pentru a evita efectul obositor de "flicker" vizual.
- **Extracția Semantică a Benzilor (Feature Extraction):**
 - **BASS (~0-250 Hz):** Controlează macro-structura și "Kick-ul" (~0-170 Hz).
 - **MID (~250-2000 Hz):** Influențează densitatea vizuală.
 - **TREBLE / HIGH (~2000+ Hz):** Direcționează efectele ascuțite (Snare, Glitch).

- **Implementare:** Fragment Shader injectat prin `FractalPostProcessService`.
- **Comunicarea CPU-GPU (Uniforms):**
 - `uBass`, `uTreble`, `uKick`: Valori trimise la fiecare cadru (Render Loop) direct din analiza FFT.
 - `uTime`: Pentru evoluția continuă a zgomotului matematic.
- **Sursa de Inspirație:** Model matematic bazat pe <https://www.shadertoy.com/view/ldfGWn>
- **Suport WebXR Nativ:**
 - În modul stereoscopic (VR), fiecare ochi primește o randare corectă.
 - Utilizarea `uInverseViewProj` (Matricea inversă de View-Projection) pentru a alinia proiecția fractală cu mișcările capului (6DoF).

- **Performanța primează în XR:** Trecerea de la 3 FPS (Editor, Geometrie complexă) la stabilizarea la 75 FPS prin abordarea "Code-First" și mutarea calculelor grele în Fragment Shader.
- **Modularitate ECS:** Dezactivarea `LevelStreamingSystem` s-a putut face printr-o singură linie de cod (`enabled = false`) datorită arhitecturii decuplate.

Sound Design – Patch-ul de Pure Data (Pd)

- **Dezvoltare Paralelă:** Crearea unui sintetizator custom utilizând mediul vizual de programare audio **Pure Data**.
- **Procesarea Semnalului:**
 - Preia date brute din fișiere .mid.
 - Transformă trigger-ele MIDI într-un spectru de sunet cromatic.
- **Integrarea în Babylon.js:** Sunetul generat procedural a fost înregistrat și exportat (.wav/.mp3), asigurând o corelație matematică perfectă cu **AnalyserNode**.

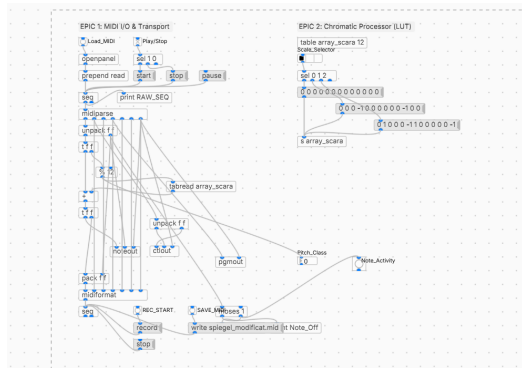


Figura: Interfața Pure Data - Patch de Sinteză

Vă mulțumesc!

Întrebări?